

AI Job Search & Application Assistant

Your LangGraph learning roadmap — built one small step at a time

How to read this document

This is the full map of the project, from start to finish. Each milestone shows: the goal (what and why), what we actually build, and its current status.

- [DONE] — already built and working, by you.
- [NEXT] — what we build next time.
- [UPCOMING] — later milestones, not started yet.

Nothing here is fixed in stone — we can reorder or adjust as we go. This is just so you can see the whole shape of the project in your mind.

Milestone 0: Setup — connect to a free LLM

[DONE] Completed

Goal: Prove your API key and Python environment actually work, with the smallest possible script.

What we build:

- Signed up for a free Groq API key.
- Created project folder + virtual environment.
- Stored the key safely in a `.env` file (never in code).
- Wrote `test.py`: sent one message to the model, printed the reply.

Why it matters: *Every later step depends on this working. No point building on a broken connection.*

Milestone b: Basic job search — real results

[DONE] Completed

Goal: Call a free job-search API and get real postings back — before touching LangGraph at all.

What we build:

- Compared free options (Adzuna, JSearch, Jooble) and chose JSearch for broad platform coverage.
- Got a free RapidAPI key, found the correct endpoint (`search-v2`).
- Wrote `job_search.py`: sent a query, printed jobs (title, company, link).
- Debugged real errors yourself (wrong endpoint path, wrong dictionary key).

Why it matters: *Confirms the data source works on its own, so later when it's wrapped inside LangGraph, you already trust it.*

Milestone b2: Wrap job search into your first LangGraph node

[DONE] Completed

Goal: Turn the plain script into a LangGraph 'node' — the basic building block of the whole project.

What we build:

- Defined JobSearchState (the shared notebook / baton passed between steps).
- Turned search code into search_jobs_node(state) -> state.
- Built a StateGraph with one node, entry point, and an edge to END.
- Compiled and ran it with app.invoke(...) — got real jobs through the graph.

Why it matters: *This is the core LangGraph pattern (state in, node processes, state out) that every future node will follow.*

Milestone a: Resume upload + understanding

[NEXT] Up next

Goal: Add a node that reads a resume and pulls out the useful parts (skills, experience, roles) so it can be compared to jobs later.

What we build:

- Add a 'resume' field to the shared state.
- Build a node that sends resume text to the LLM and asks it to summarize skills/experience.
- Test it on a sample resume (your real resume only when you're ready to share it).

Why it matters: *The assistant can't score job matches without first understanding what you actually offer.*

Milestone c: Score job matches with reasoning

[NEXT] Upcoming

Goal: Add a node that compares each job posting to your resume summary and gives a match score + short reason.

What we build:

- Build a node that loops through jobs and asks the LLM to score each one (e.g. 0-100) with a one-line reason.
- Store scores back into the shared state next to each job.

Why it matters: *This turns a plain list of jobs into a ranked, personalized list — the first real 'assistant' behavior.*

Milestone d: Retry / broaden search when matches are weak

[NEXT] Upcoming

Goal: Teach the graph to notice when scores are low and automatically try a broader search instead of giving up.

What we build:

- Add a conditional edge: if best score is below a threshold, loop back to the search node with a broader query.
- Add a simple limit so it doesn't loop forever.

Why it matters: *This is your first branching / looping graph — LangGraph's real advantage over a plain script.*

Milestone e: Application tracking

[NEXT] Upcoming

Goal: Let you mark a job as applied, and track its status over time (applied / interview / rejected / no response). No auto-apply, ever.

What we build:

- Add simple functions/nodes to update a job's status.
- Keep this fully manual — you decide, the assistant never submits anything.

Why it matters: *This is the 'assistant', not 'autopilot' — matches your requirement that nothing auto-applies.*

Milestone f: Persistence with SQLite

[NEXT] Upcoming

Goal: Save tracked jobs and statuses to a real local database, so nothing is lost between runs.

What we build:

- Set up a small SQLite database (free, built into Python).
- Save/load job + status data instead of keeping it only in memory.

Why it matters: Right now everything disappears when the script ends. This makes the assistant actually useful day to day.

Milestone g: Long-term preference memory

[NEXT] Upcoming

Goal: Let the assistant learn from which jobs you apply to vs skip, to improve future recommendations.

What we build:

- Track patterns from your applied/rejected choices.
- Feed a short summary of your preferences back into the scoring node.

Why it matters: This makes the assistant adapt to you personally over time, not just repeat the same generic scoring.

Milestone h: Streaming progress updates

[NEXT] Upcoming

Goal: Show live progress while the graph runs (e.g. 'Searching...', 'Scoring job 3 of 12...') instead of a silent wait.

What we build:

- Use LangGraph's streaming feature to print updates as each node runs.

Why it matters: Better experience — you see what the assistant is doing, especially once it involves several steps and API calls.

Milestone i: Observability / tracing

[NEXT] Upcoming

Goal: Set up a free way to see exactly what happened inside a graph run (which node ran, what it returned, any errors).

What we build:

- Connect a free tracing tool (e.g. LangSmith free tier) to your graph.
- Use it to debug and understand runs, especially once the graph has loops and branches.

Why it matters: As the graph grows more complex, print statements aren't enough — you'll want a proper trace view.

Ground rules we're following

- Every tool/API used must be free — no paid keys required.
- No auto-applying to jobs, ever — you stay in control.
- One small step at a time — you write the code, I guide.
- No deadline — depth over speed.
- Real resume/personal data only shared when a milestone actually needs it, and only after you agree.